



香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

ELEC 3310

Digital Fundamentals and System Design

Wenchao Miao

Outline Lecture 8 Finite State Machines Analysis



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

1 Learning Outcomes

2 Analysis of Sequential Circuits

3 State Reduction

4 Analysis with JK Flip-Flops

5 Analysis with T Flip-Flops

Main Content

1. Learning Outcomes

2. General Form of a Sequential Circuit

1. **Analyze** (**reverse engineer**) a given sequential logic circuit (FSM)

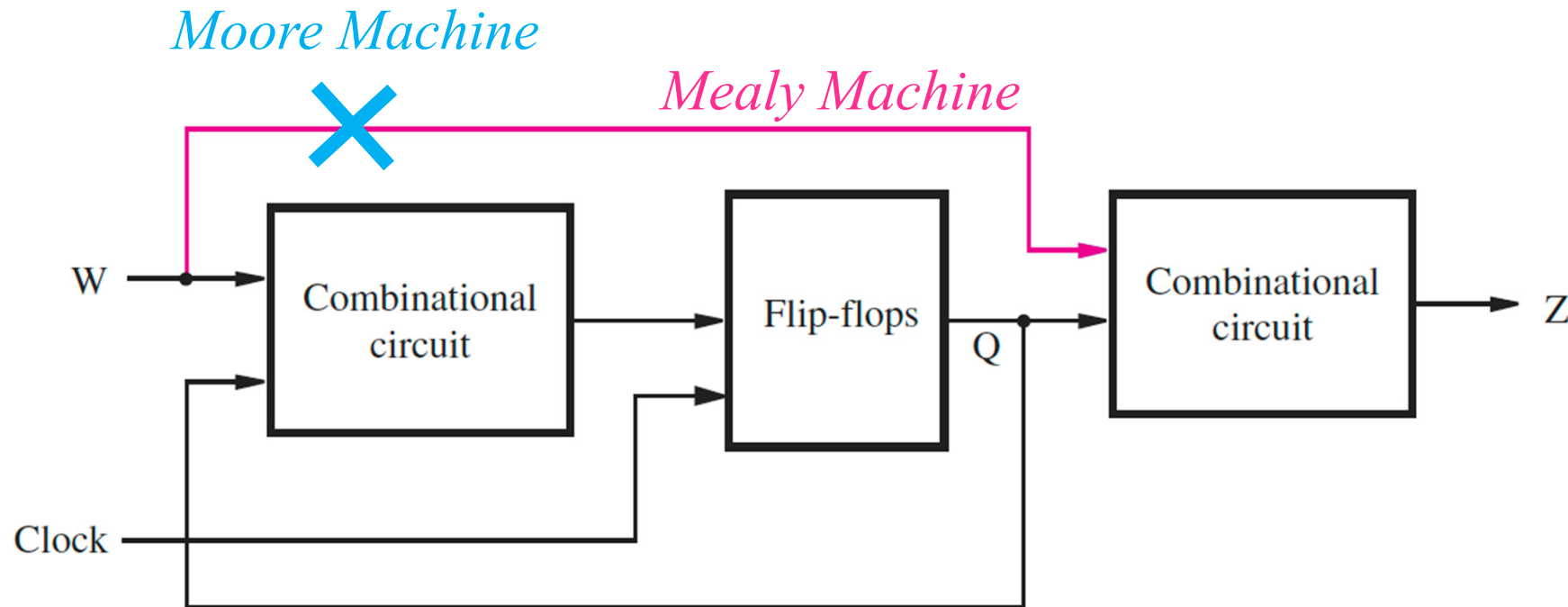
Analyze = opposite of design

2. **Derive a state diagram and state transition table for a given sequential logic circuit**
3. **Describe steps applied to reverse engineer a given sequential circuit**
4. **Describe the operation of JK and T flip flops**
5. **Describe the input equations of JK and T flip flops**
6. **Explain the difference between D, JK, and T flip flops**

General Form of a Sequential Circuit



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY



1. Obtain specification of the desired circuit.
 2. Create a state diagram from specification.
 3. Create a state table from state diagram.
 4. Perform state minimization.
 5. Perform state assignment.
 6. Derive the next-state and output logic expressions.
 7. Implement circuit described by logic.
- Steps 4 & 5 are not in counter design procedure*

A state table consists of 4 sections

– **Present State (PS), Inputs, Next State (NS), and outputs**

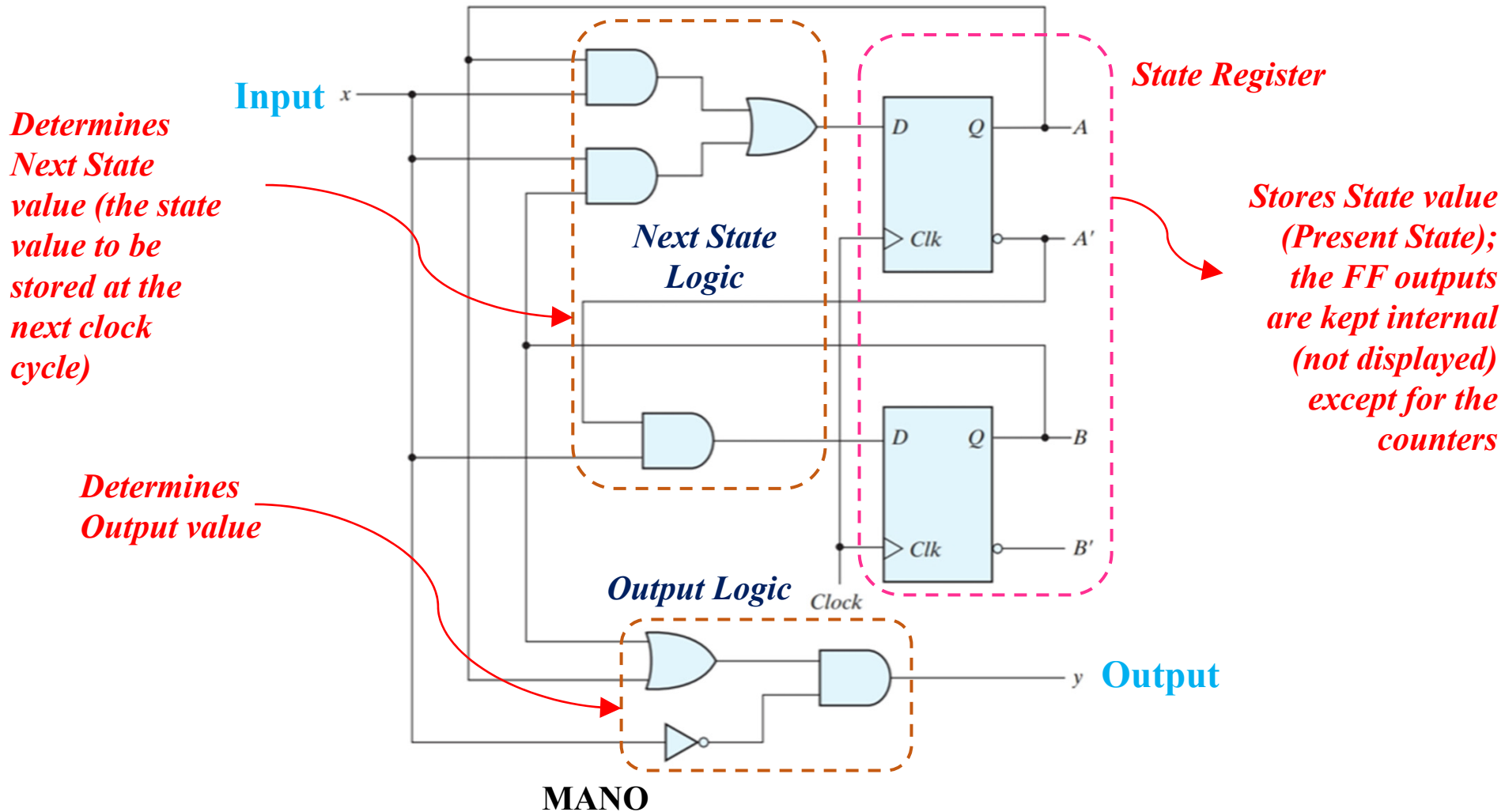
- 1. Treat PS and Inputs as “inputs” and fill the columns by listing all binary combinations (like truth table)**
- 2. Derive State and output equations from the circuit**
- 3. Use State equations to fill NS column(s)**
- 4. Use output equations to fill output column(s)**
- 5. Draw State diagram based on the table**

Analysis (Reverse Engineering)

Approach: **Logic/circuit diagram** → Logic Expressions → State Table → State Diagram

Analyze (reverse engineer) the following FSM. What does it do ?

This circuit has single input (x) and single output (y)

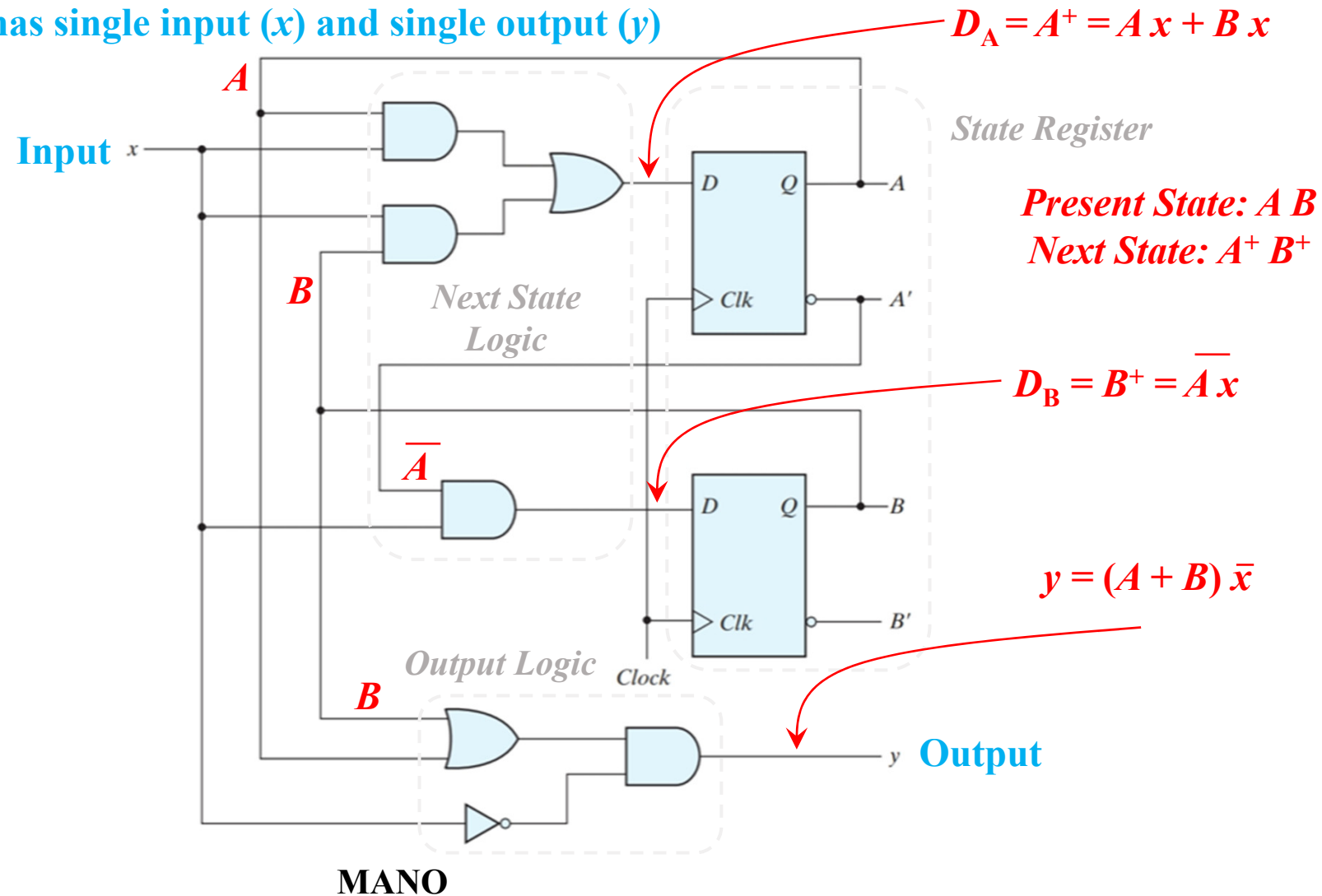


Analysis (Reverse Engineering)

Approach: Logic/circuit diagram → **Logic Expressions** → State Table → State Diagram

Analyze (reverse engineer) the following FSM. What does it do ?

This circuit has single input (x) and single output (y)



Approach: Logic/circuit diagram → Logic Expressions → **State Table** → State Diagram

Use logic expressions to fill the State Table

Treat it as a 3-input TRUTH table

Inputs: Present State (A , B), and x (circuit input)

Outputs: Next State values (A^+ , B^+) and y (circuit output)

Present State: $A B$

Next State: $A^+ B^+$

Inputs

				Results		
Present State			Input	Next State		Output
A	B		x	A^+	B^+	y
0	0	0	0	0	0	0
1	0	0	1	0	1	0
2	0	1	0	0	0	1
3	0	1	1	1	1	0
4	1	0	0	0	0	1
5	1	0	1	1	0	0
6	1	1	0	0	0	1
7	1	1	1	1	0	0

$$D_A = A^+ = A x + B x$$

$$D_B = B^+ = \bar{A} x$$

$$y = (A + B) \bar{x}$$

Approach: Logic/circuit diagram → Logic Expressions → **State Table** → State Diagram

Present State		Input x	Next State		Output y
A	B		A^+	B^+	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

Alternative format

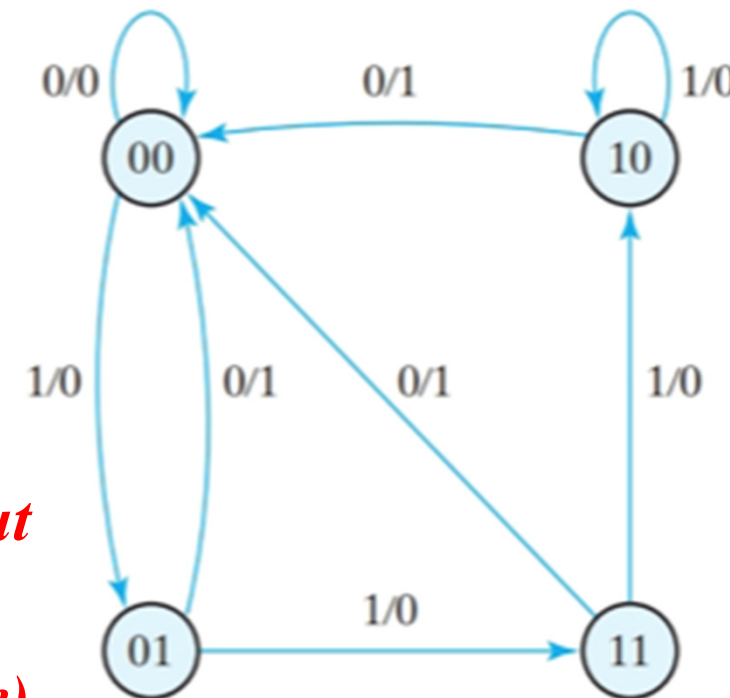
State Table can be rearranged like this:

Present State A B		Next State				Output	
		$x = 0$		$x = 1$		$x = 0$	$x = 1$
		A^+	B^+	A^+	B^+	y	y
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0

Approach: Logic/circuit diagram → Logic Expressions → State Table → **State Diagram**

Present State		Next State				Output	
		$x = 0$		$x = 1$		$x = 0$	$x = 1$
A	B	A^+	B^+	A^+	B^+	y	y
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0

From table to diagram



Based on the state diagram, can you tell what the circuit does ?

Detects sequence 10

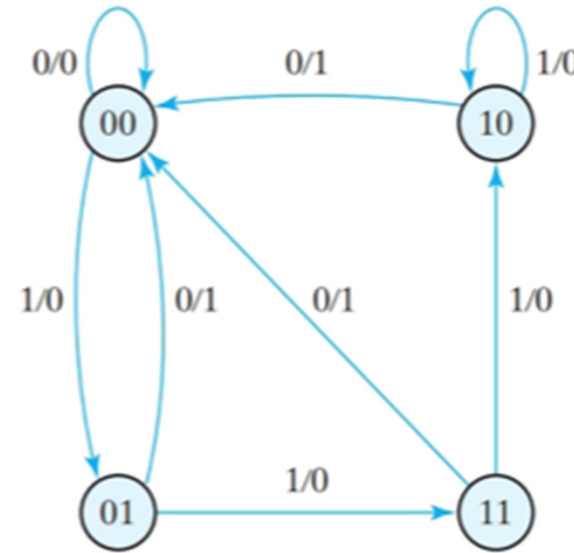
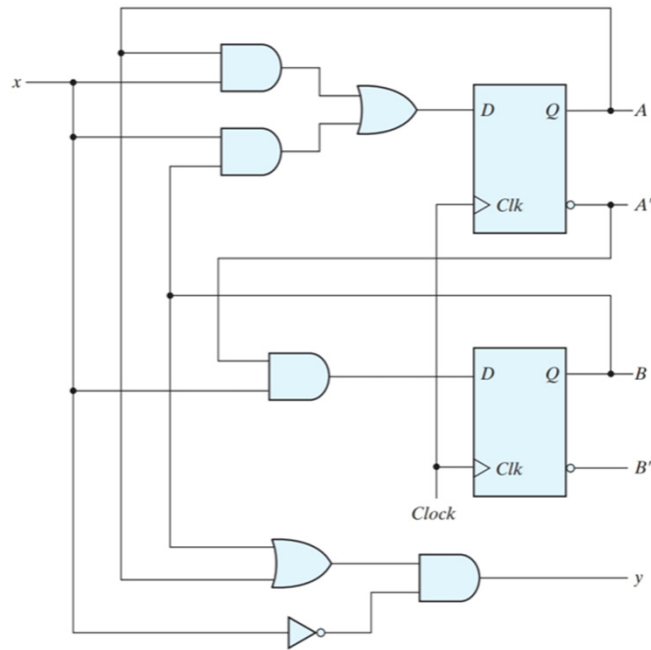
Is this Mealy or Moore machine ?

Mealy: output may change with input

Can this circuit be made simpler

Yes using only one FF (see next slide)

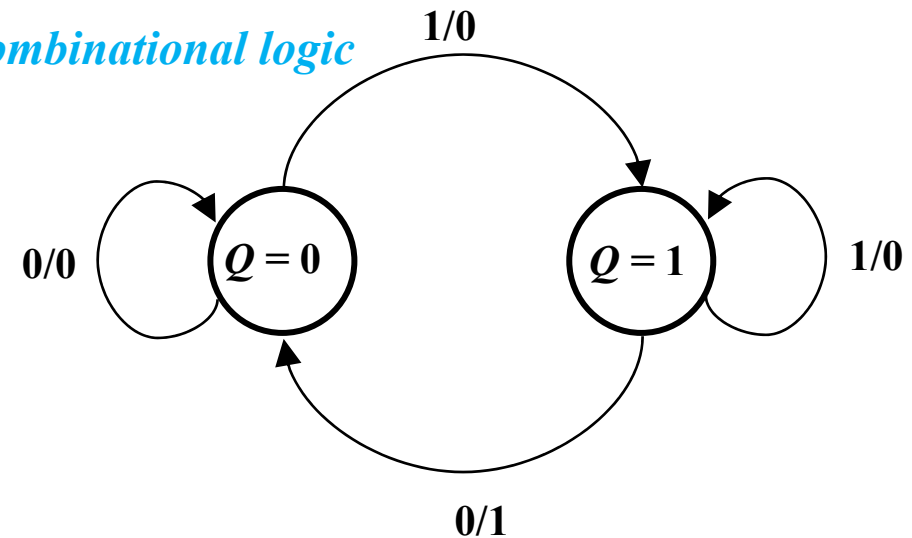
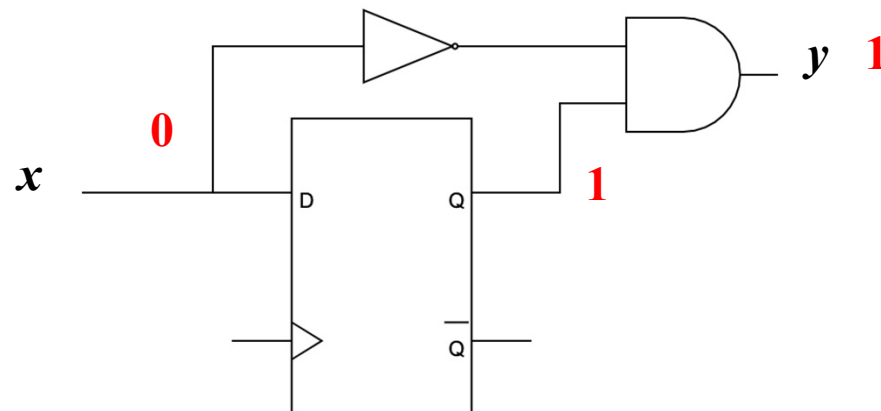
Analysis (Reverse Engineering)



This FSM uses unnecessary states and unnecessary FF. States 01, 10, and 11 are equivalent (only one is sufficient).

Next we will see how to detect and eliminate redundant states

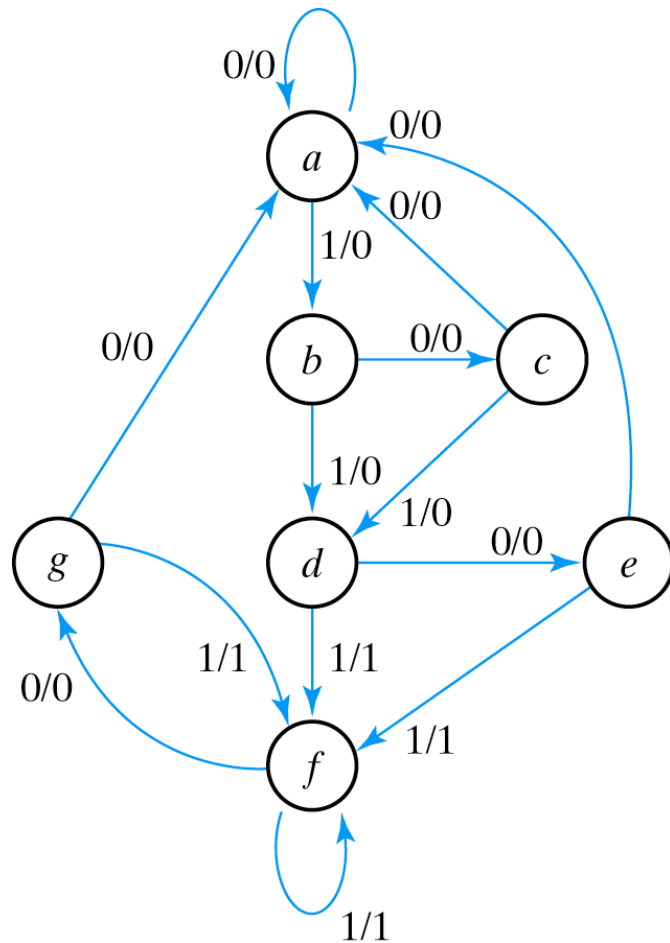
We eliminate 1 FF and use only a single inverter for combinational logic
Simplified sequence 10 detector



- Two FSM are equivalent if both produce the same output sequence given the same input sequence.
- The internal 'states' are not relevant from the user point of view. The user may not see the state values (except counters)
- Try to find an equivalent state machine, with fewer internal states (i.e. simpler? fewer flip-flops? perhaps...)
- The principle of FSM state reduction is based on *identifying and merging* equivalent states.
- "Two internal states are said to be equivalent if both give exactly the same output for each input combination and send the circuit to either the same state or to an equivalent state"

Example: State Reduction

- Use row matching method on state table



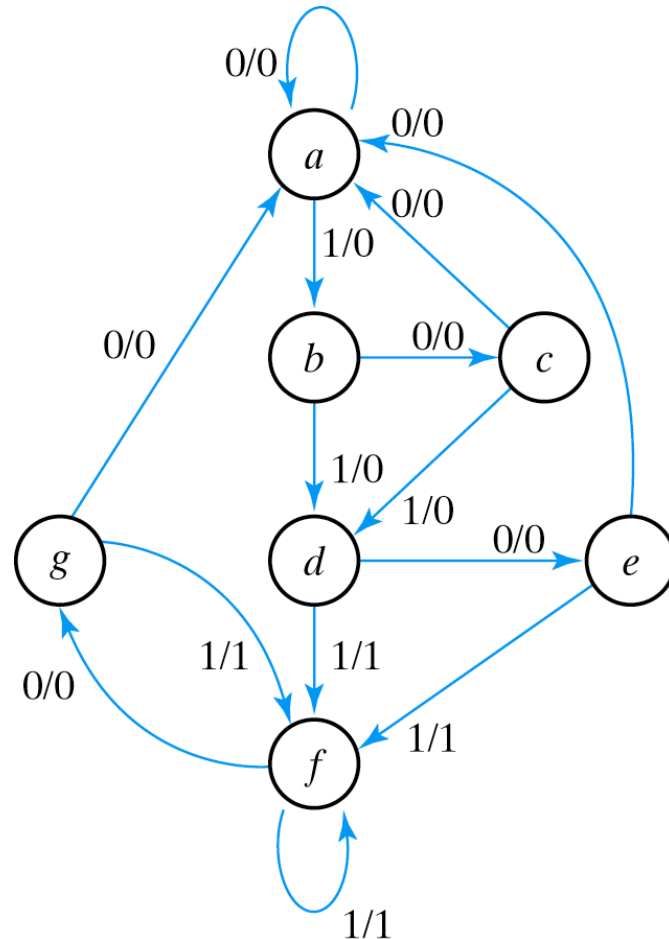
State e and g are equivalent because for all input cases, both produce identical outputs and send FSM to the same next state

State g can be removed

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
<i>a</i>	<i>a</i>	<i>b</i>	0	0
<i>b</i>	<i>c</i>	<i>d</i>	0	0
<i>c</i>	<i>a</i>	<i>d</i>	0	0
<i>d</i>	<i>e</i>	<i>f</i>	0	1
<i>e</i>	<i>a</i>	<i>f</i>	0	1
<i>f</i>	<i>g</i>	<i>f</i>	0	1
<i>g</i>	<i>a</i>	<i>f</i>	0	1

Example: State Reduction

- Use row matching method on state table



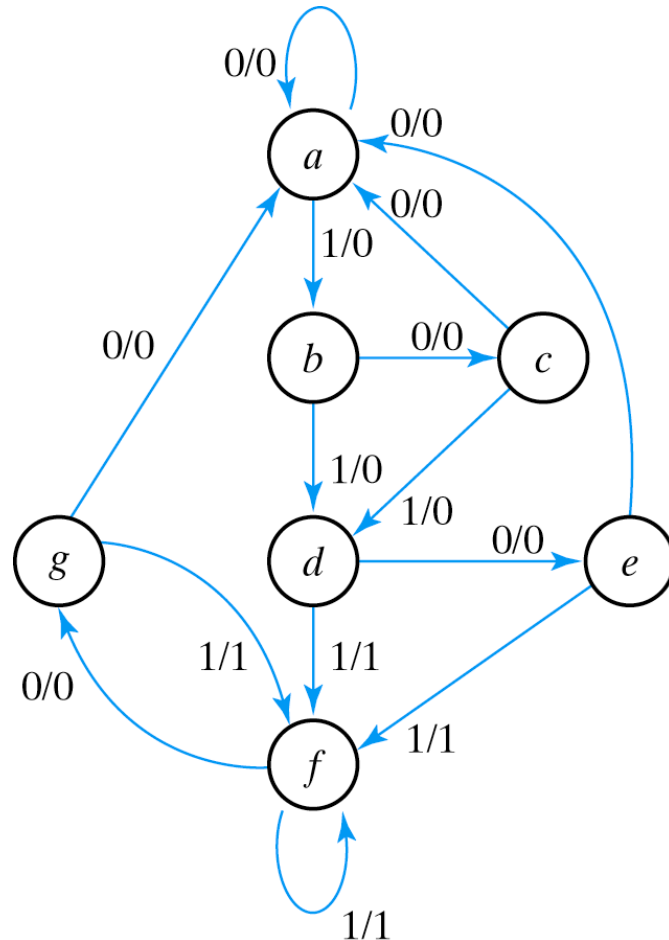
State e and g are equivalent because for all input cases, both produce identical outputs and send FSM to the same next state

State g can be removed

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
<i>a</i>	<i>a</i>	<i>b</i>	0	0
<i>b</i>	<i>c</i>	<i>d</i>	0	0
<i>c</i>	<i>a</i>	<i>d</i>	0	0
<i>d</i>	<i>e</i>	<i>f</i>	0	1
<i>e</i>	<i>a</i>	<i>f</i>	0	1
<i>f</i>	<i>e</i>	<i>f</i>	0	1
<i>g</i>	<i>a</i>	<i>f</i>	0	1

Example: State Reduction

- Use row matching method on state table



State d and f are equivalent because for all input cases, both produce identical outputs and send FSM to the same next state

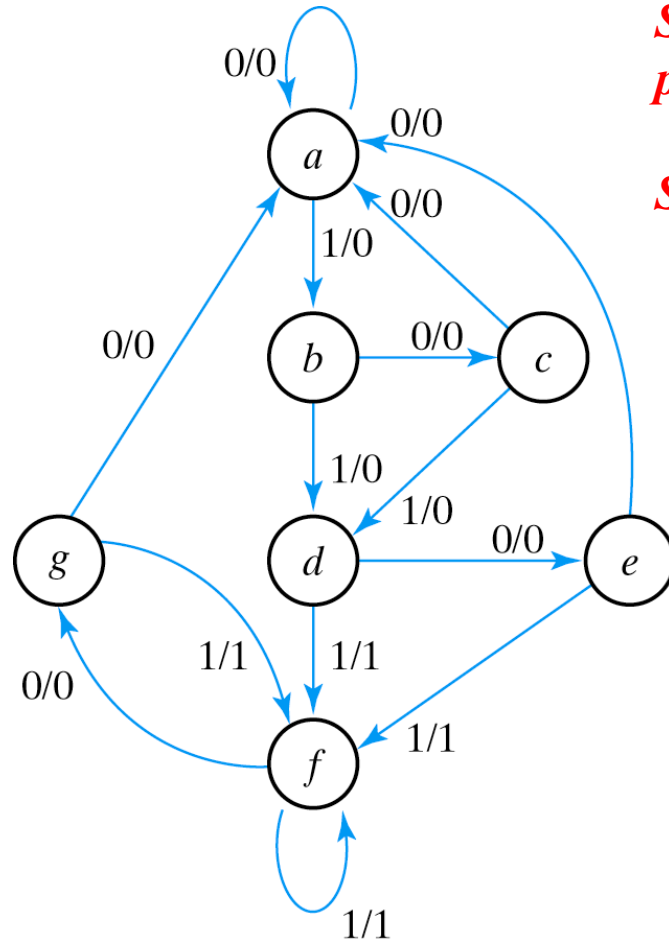
State f can be removed

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
<i>a</i>	<i>a</i>	<i>b</i>	0	0
<i>b</i>	<i>c</i>	<i>d</i>	0	0
<i>c</i>	<i>a</i>	<i>d</i>	0	0
<i>d</i>	<i>e</i>	<i>f</i>	0	1
<i>e</i>	<i>a</i>	<i>f</i>	0	1
<i>f</i>	<i>e</i>	<i>f</i>	0	1

Example: State Reduction



- Use row matching method on state table



State d and f are equivalent because for all input cases, both produce identical outputs and send FSM to the same next state

State f can be removed

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	f	0	1
e	a	f	0	1
f	e	f	0	1

Example: State Reduction

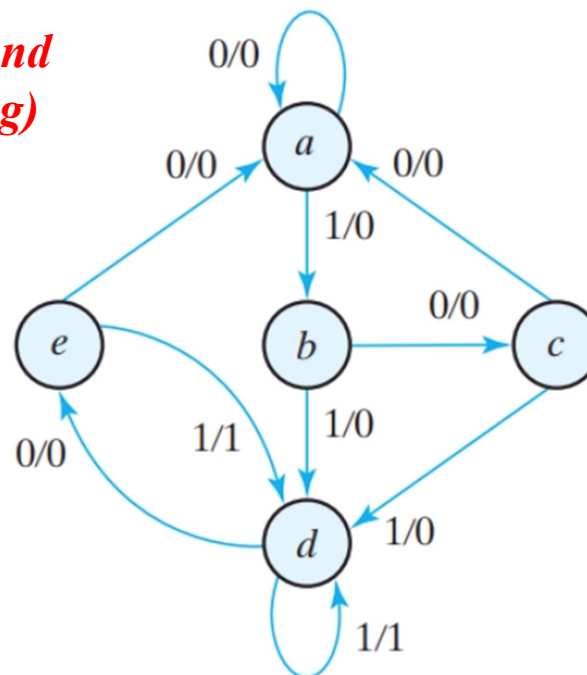
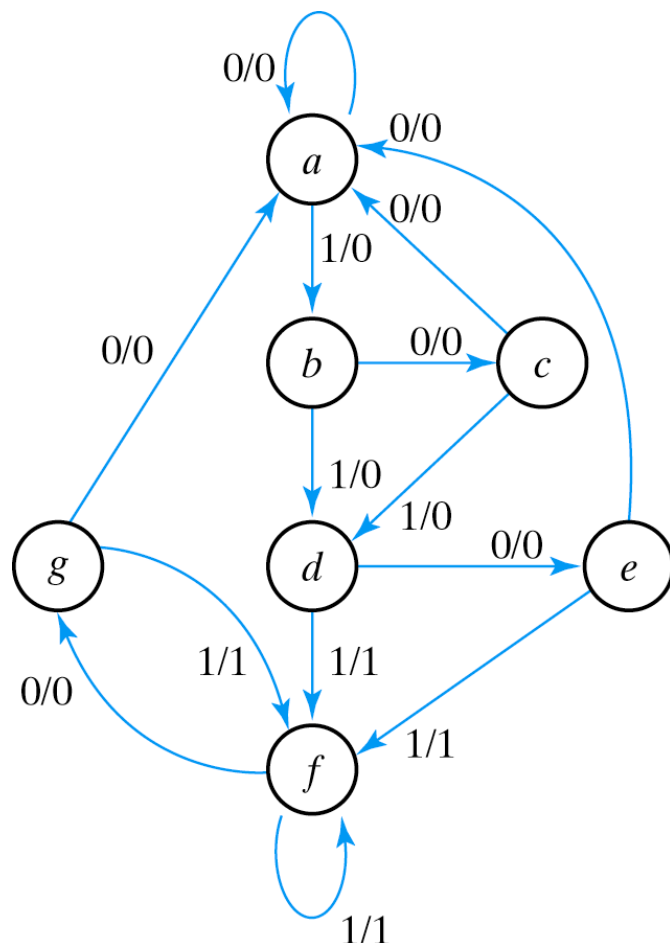
- *States c and e are NOT equivalent.*
 - *Although both have the same state transition with respect to the input, both differ in output behavior*
- *Next, check whether d and e are equivalent*
 - *Both produce the same outputs for given inputs*
 - *Both send the FSM to the same state (d) but differ in other states (a and e)*
 - *To ensure you need to check whether the states a and e are equivalent or not!*
 - *The states a and e are not equivalent because they differ in the output values*
- *Also you need to check whether the states a , b , and c are equivalent*
 - *The states a and c are equivalent if b and d are equivalent. However, the states b and d are NOT equivalent because they differ in output*
- *No further state to eliminate. We're done!*

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1
-	-	-	-	-

Example: State Reduction



Using row-matching method we identified and removed two redundant states (states f and g)



Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1
-	-	-	-	-

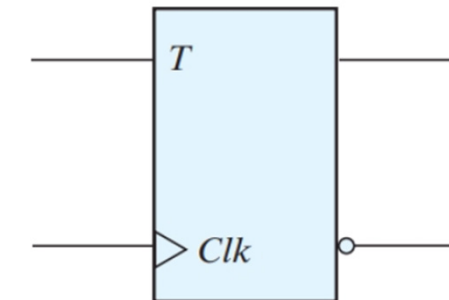
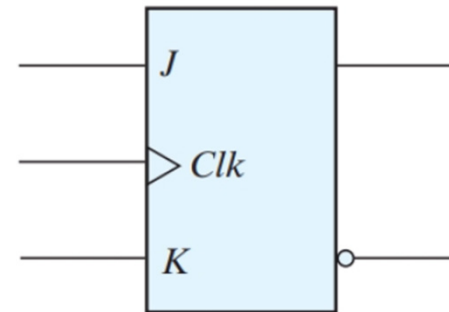
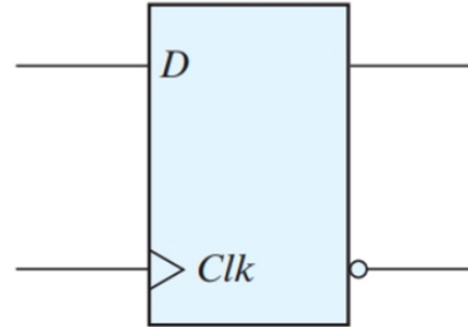
- **D Flip-flop**

Both JK and T flip-flops can be produced from D flip-flop using some logic gates

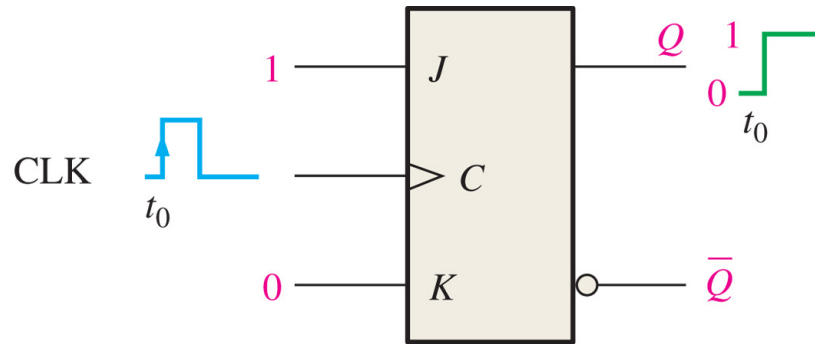
- **JK Flip-flop**

Similarly, one FF type can be converted into another FF type using some logic gates

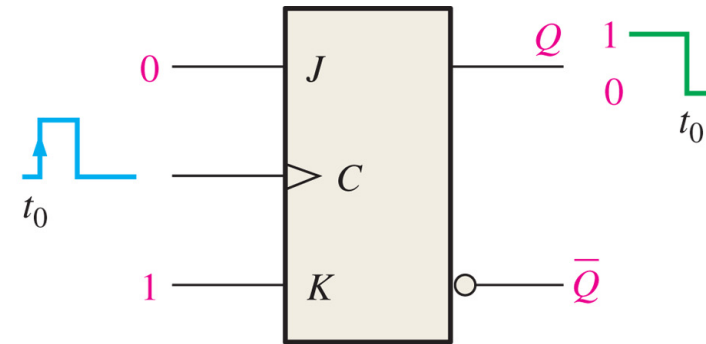
- **T Flip-flop**



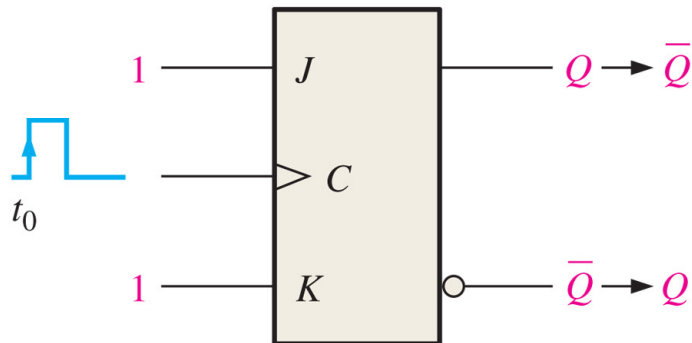
JK Flip-Flop



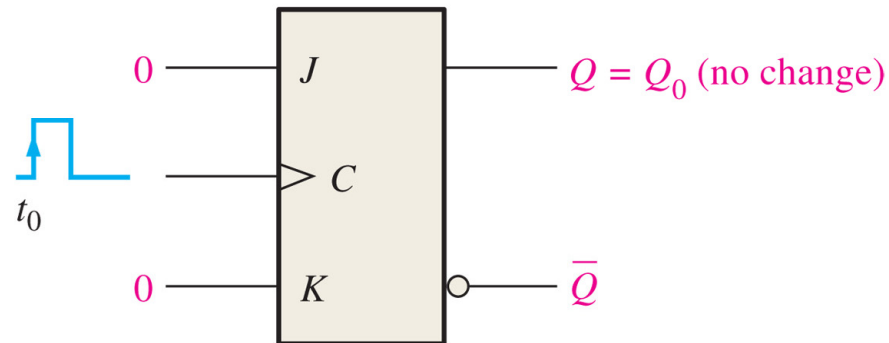
(a) $J = 1, K = 0$ flip-flop SETS on positive clock edge. (If already SET, it remains SET.)



(b) $J = 0, K = 1$ flip-flop RESETS on positive clock edge. (If already RESET, it remains RESET.)



(c) $J = 1, K = 1$ flip-flop changes state (toggle).



(d) $J = 0, K = 0$ flip-flop does not change. (If SET, it remains SET; if RESET, it remains RESET.)

THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

(a) Timing diagram for a J-K flip-flop. The diagram shows four signals: CLK, J, K, and Q over five clock cycles. CLK is a periodic square wave. J and K are control inputs that change at specific clock edges. Q is the output, which toggles when J=K=1, remains unchanged when J=K=0, resets to 0 when J=0, K=1, and sets to 1 when J=1, K=0. Vertical dashed lines mark the clock edges, and labels below the Q signal indicate the action: Toggle, No change, Reset, Set, Set.



Characteristic Table

JK Flip-Flop			
J	K	$Q(t + 1) = Q^+$	
0	0	$Q(t)$	No change
0	1	0	Reset
1	0	1	Set
1	1	$Q'(t)$	Complement

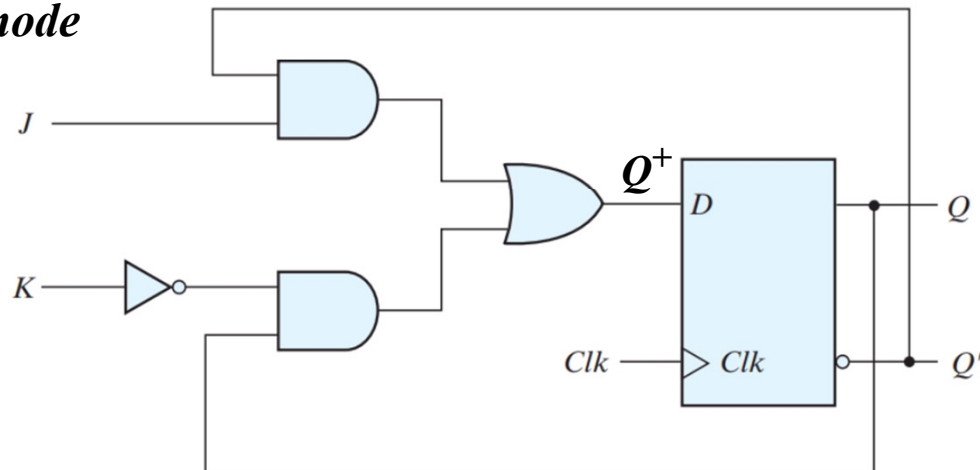
Expand into 3-input truth table where present state Q is one of the inputs and next state Q^+ is the output. And then write Q^+ using minterms or maxterms

$$Q^+ = \bar{J}\bar{K}Q + J\bar{K}\bar{Q} + J\bar{K}Q + JK\bar{Q}$$

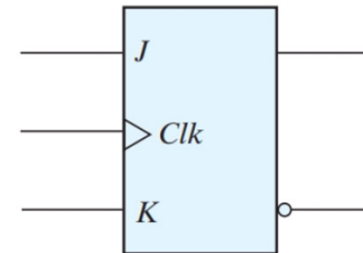
$$= J\bar{Q} + \bar{K}Q$$

J	K	Q	Q^+
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Complement also known as Toggle mode

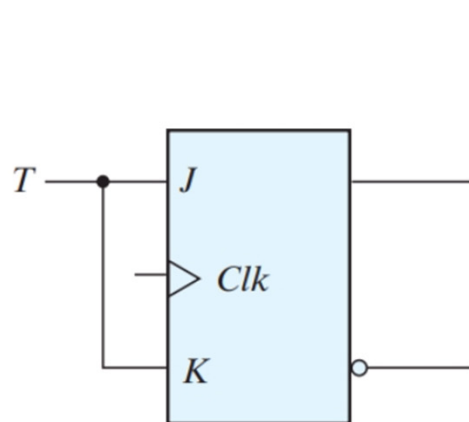


(a) Circuit diagram

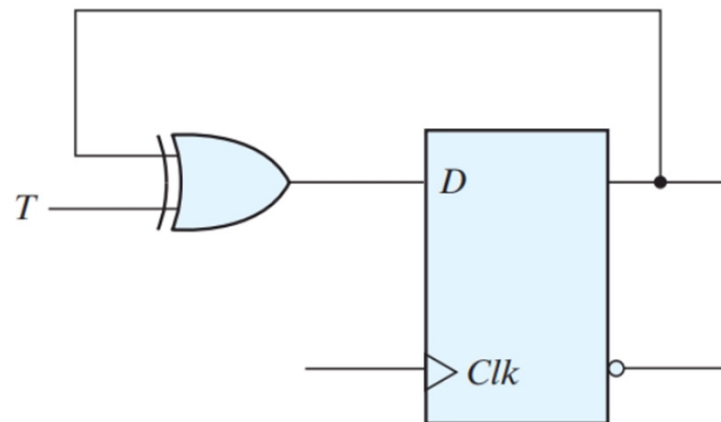


(b) Graphic symbol

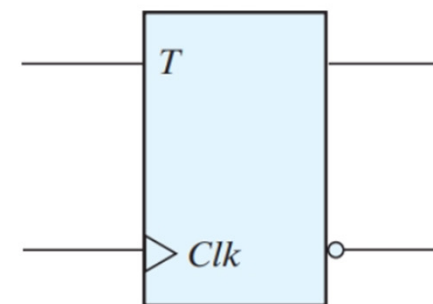
*T flip-flop can be implemented using JK FF by shorting J and K inputs
It can be also implemented using D FF and an XOR gate*



(a) From JK flip-flop



(b) From D flip-flop



(c) Graphic symbol

T Flip-Flop		
T	$Q(t + 1) = Q^+$	
0	$Q(t)$	No change
1	$Q'(t)$	Complement

Expand into 2-input truth table where present state Q is one of the inputs and next state Q^+ is the output. And then write Q^+ using minterms or maxterms

T	Q	Q^+
0	0	0
0	1	1
1	0	1
1	1	0

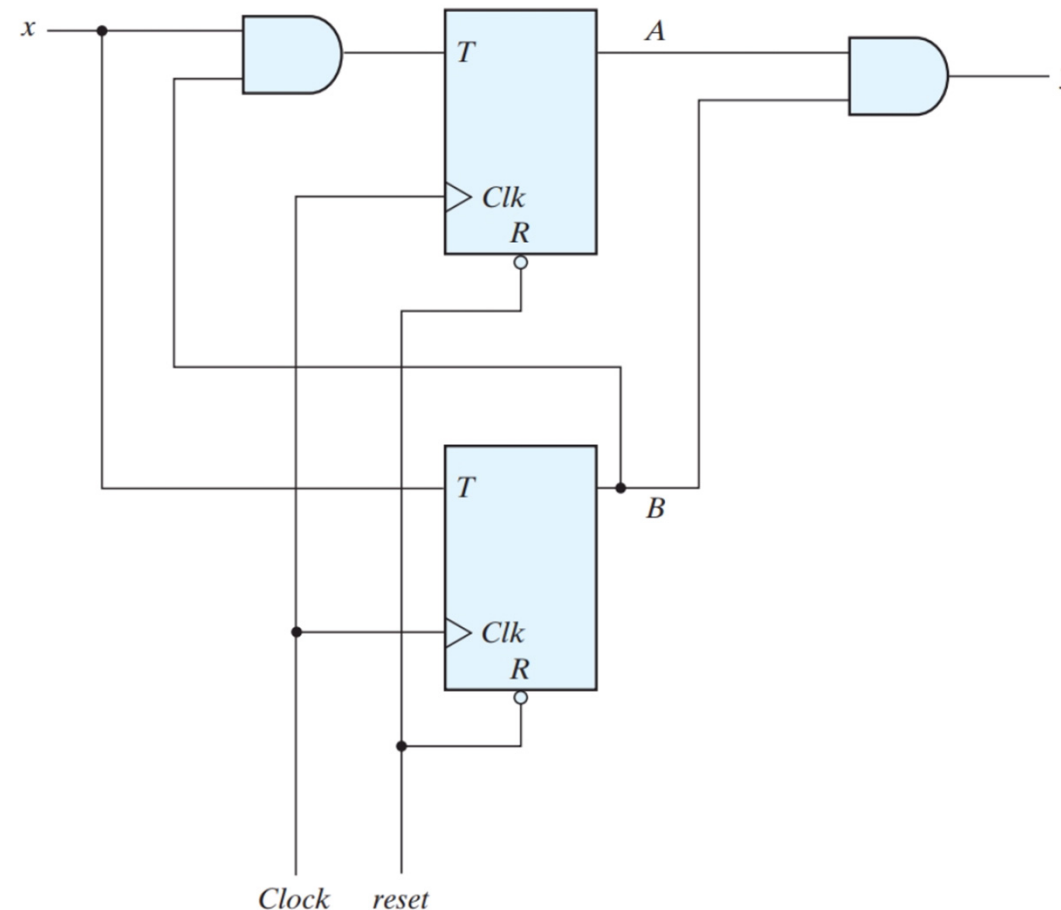
$$Q^+ = T \oplus Q$$

Complement also known as Toggle mode

Approach: **Logic/circuit diagram** → Logic Expressions → State Table → State Diagram

Analyze (reverse engineer) the following FSM. What does it do ?

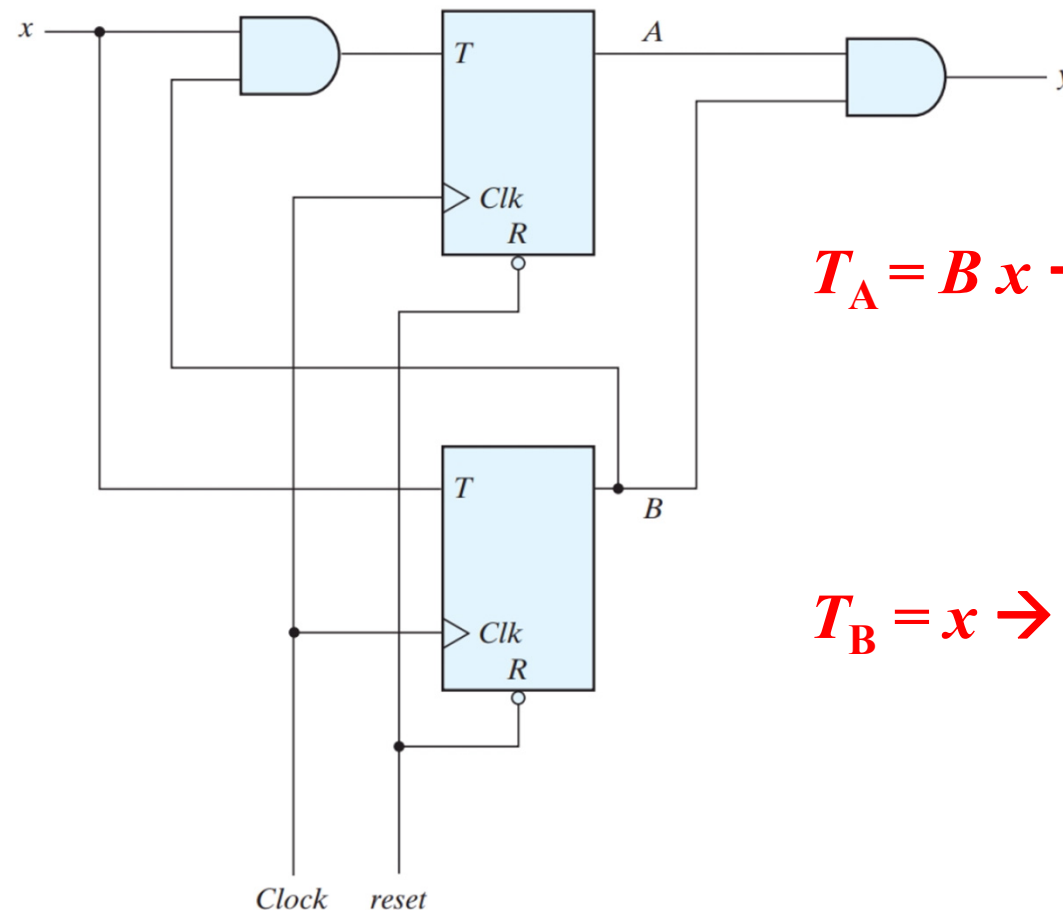
This circuit has single input (x) and single output (y)



Approach: Logic/circuit diagram → **Logic Expressions** → State Table → State Diagram

Analyze (reverse engineer) the following FSM. What does it do ?

This circuit has single input (x) and single output (y)



$$Q^+ = T \oplus Q = T \bar{Q} + \bar{T} Q$$

$$y = A B$$

$$T_A = B x \rightarrow A^+ = B x \bar{A} + \bar{B} x A$$

$$= \bar{A} B x + A \bar{B} + A \bar{x}$$

$$T_B = x \rightarrow B^+ = x \oplus B$$

Approach: Logic/circuit diagram → Logic Expressions → **State Table** → State Diagram

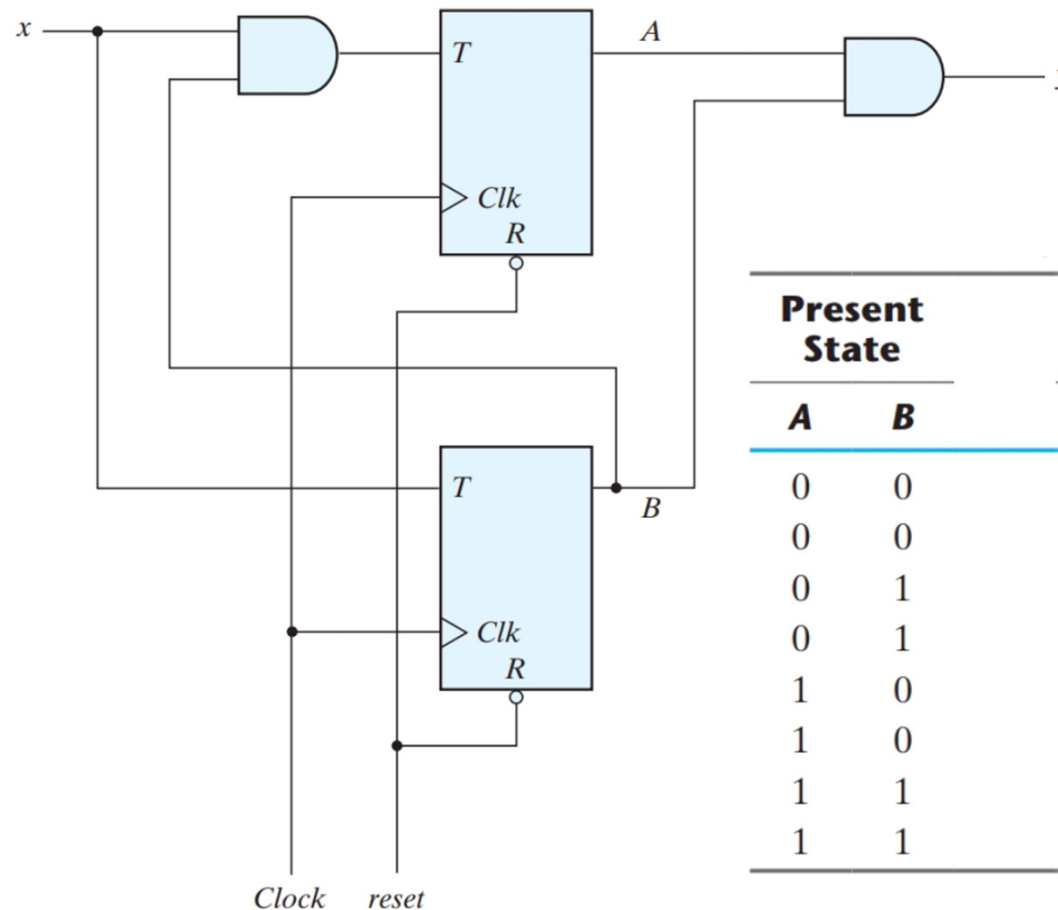
Analyze (reverse engineer) the following FSM. What does it do ?

This circuit has single input (x) and single output (y)

$$y = A B$$

$$A^+ = \bar{A} B x + A \bar{B} + A \bar{x}$$

$$B^+ = x \oplus B$$



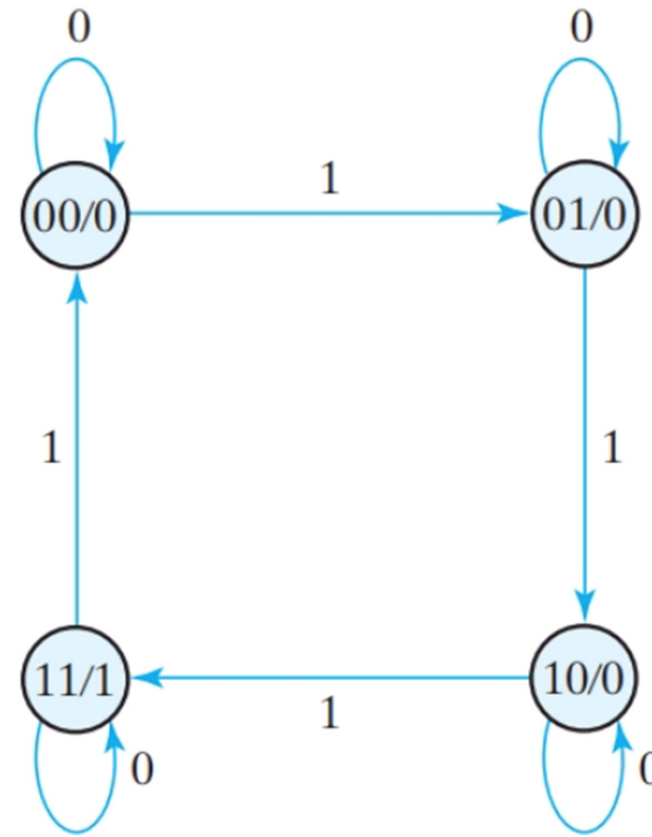
Present State		Input x	Next State		Output y
A	B		A^+	B^+	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1

Approach: Logic/circuit diagram → Logic Expressions → State Table → **State Diagram**

Analyze (reverse engineer) the following FSM. What does it do ?

Present State		Input x	Next State		Output y
A	B		A^+	B^+	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1

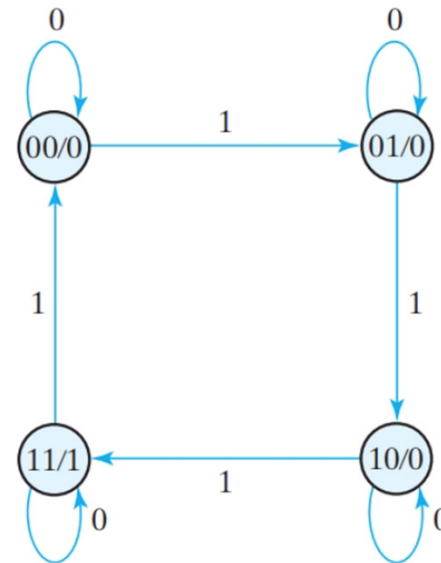
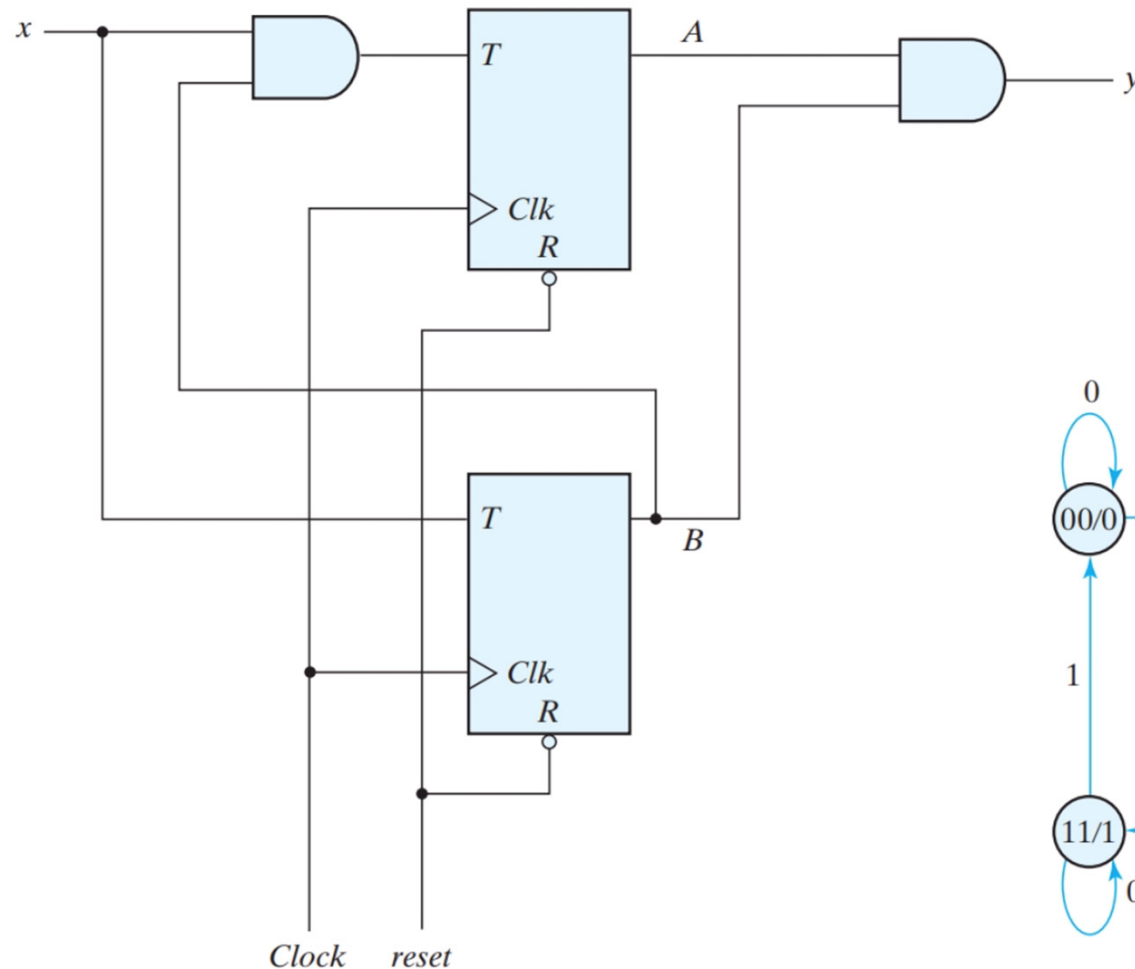
This circuit has single input (x) and single output (y)



Analysis (Reverse Engineering)

Approach: Logic/circuit diagram → Logic Expressions → State Table → **State Diagram**

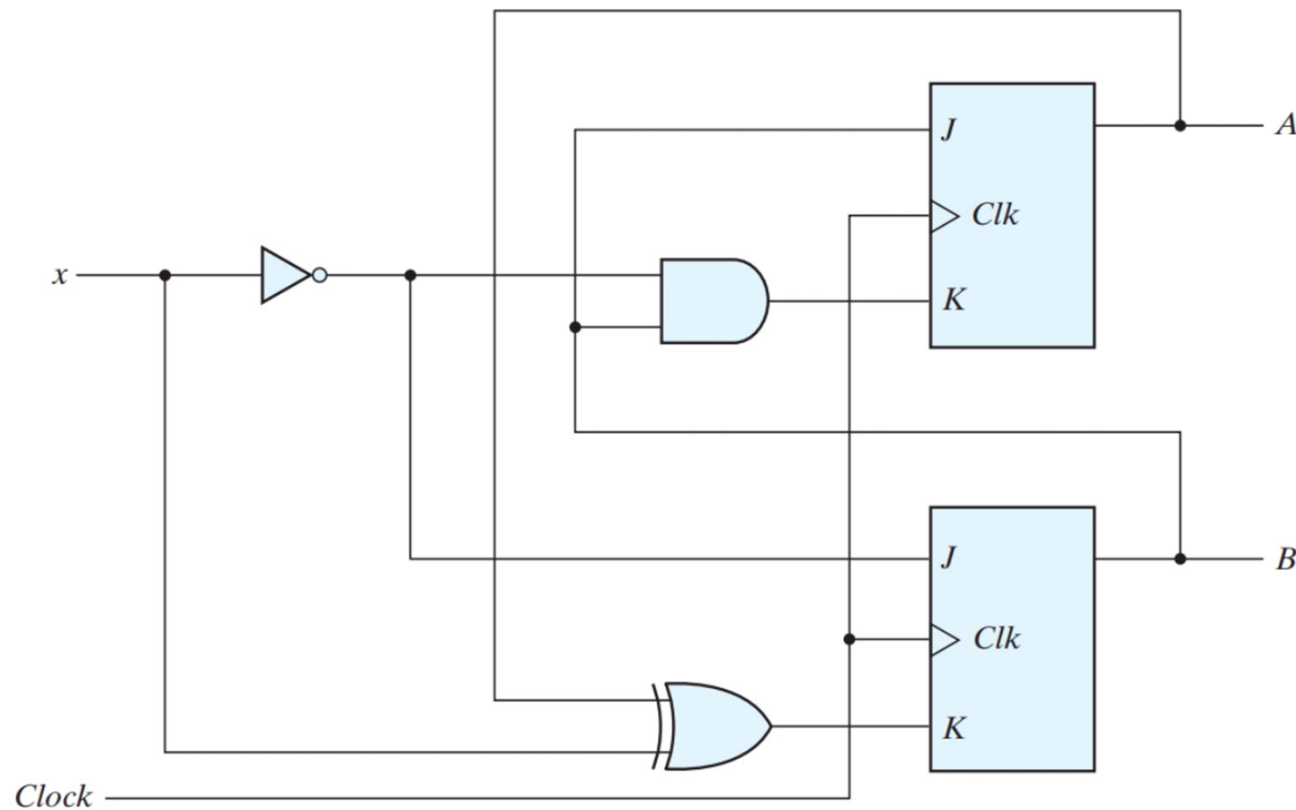
Analyze (reverse engineer) the following FSM. What does it do ?



Approach: **Logic/circuit diagram** → Logic Expressions → State Table → State Diagram

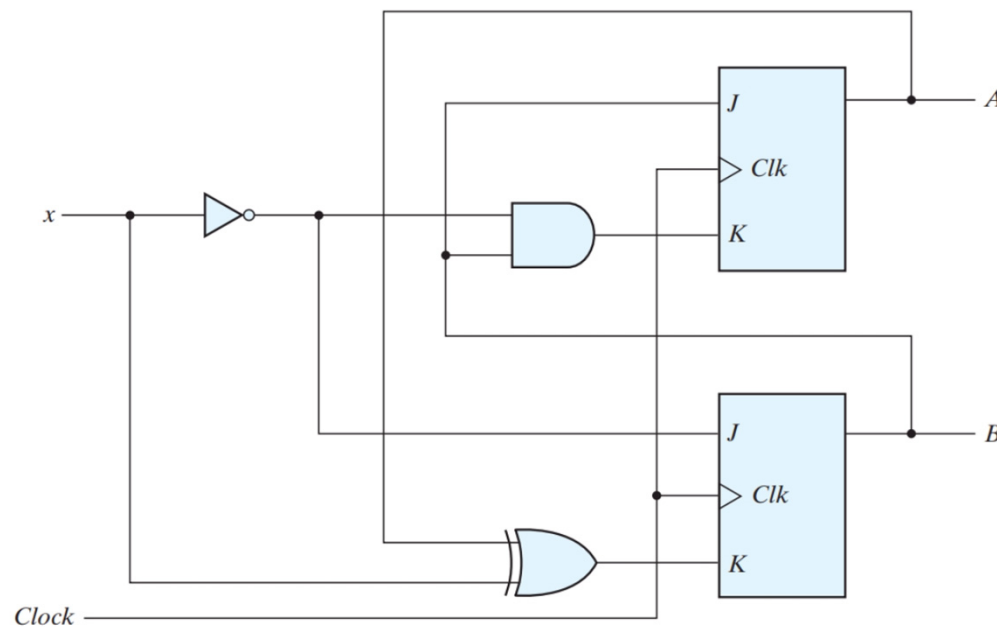
Analyze (reverse engineer) the following FSM. What does it do ?

This circuit has single input (x) and NO output



Approach: Logic/circuit diagram → **Logic Expressions** → State Table → State Diagram

$$Q^+ = J \bar{Q} + \bar{K} Q$$



$$J_A = B \quad K_A = \bar{x} B$$

$$A^+ = J_A \bar{A} + \bar{K}_A A$$

$$A^+ = B \bar{A} + \bar{x} B A$$

$$A^+ = B \bar{A} + x A + \bar{B} A$$

$$J_B = \bar{x} \quad K_B = A \oplus x$$

$$B^+ = J_B \bar{B} + \bar{K}_B B$$

$$B^+ = \bar{x} \bar{B} + (\bar{A} \oplus x) B$$

$$B^+ = \bar{x} \bar{B} + A B x + \bar{A} B \bar{x}$$

Approach: Logic/circuit diagram → Logic Expressions → **State Table** → State Diagram

$$A^+ = B \bar{A} + x A + \bar{B} A$$

$$J_A = B$$

$$K_A = \bar{x} B$$

$$B^+ = \bar{x} \bar{B} + A B x + \bar{A} B \bar{x}$$

$$J_B = \bar{x}$$

$$K_B = A \oplus x$$

Use state

Obtained from input equations

List all the 8 combinations

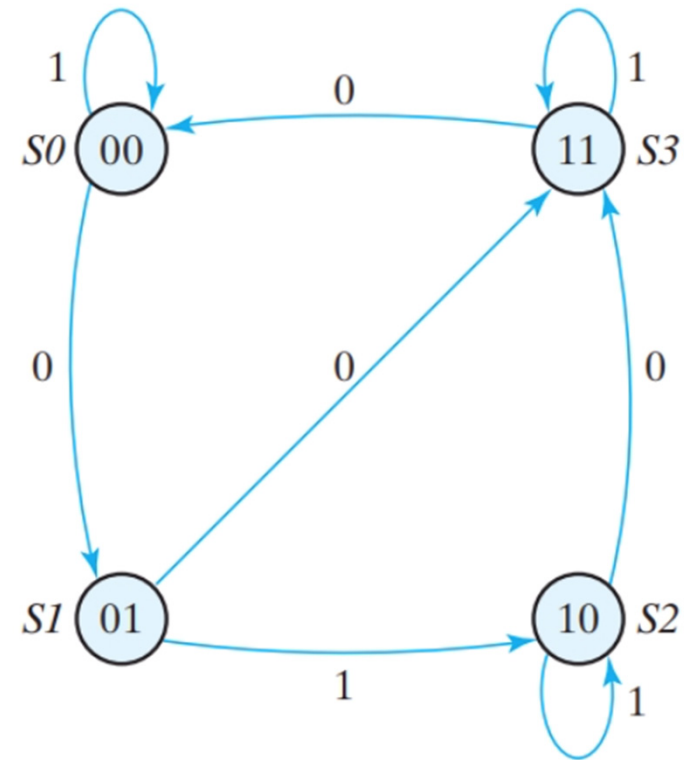
equations to fill

(not part of the state table but needed)

Present State			Input		Next State		Flip-Flop Inputs			
A	B		x		A ⁺	B ⁺	J _A	K _A	J _B	K _B
0	0		0		0	1	0	0	1	0
0	0		1		0	0	0	0	0	1
0	1		0		1	1	1	1	1	0
0	1		1		1	0	1	0	0	1
1	0		0		1	1	0	0	1	1
1	0		1		1	0	0	0	0	0
1	1		0		0	0	1	1	1	1
1	1		1		1	1	1	0	0	0

Approach: Logic/circuit diagram → Logic Expressions → State Table → **State Diagram**

Present State		Input	Next State		Flip-Flop Inputs			
A	B		A	B	J_A	K_A	J_B	K_B
0	0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	0	1
0	1	0	1	1	1	1	1	0
0	1	1	1	0	1	0	0	1
1	0	0	1	1	0	0	1	1
1	0	1	1	0	0	0	0	0
1	1	0	0	0	1	1	1	1
1	1	1	1	1	1	0	0	0





香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Thank you for listening !

